



گزارش آزمایش
آزمایشگاه مدار منطقی

آزمایش چهارم
واحد محاسبات و منطق (ALU)

اعضای گروه:

نام و نام خانوادگی	شماره دانشجویی
روناک ابوطالبی	۴۰۴۱۰۵۳۹۴
حسین مرادی	۴۰۴۱۰۶۳۶۶

تاریخ تحویل: ۱۴۰۵/۰۶/۱۱

تابستان ۱۴۰۵

فهرست مطالب

۲	۱	مقدمه و تعریف مسئله
۳	۲	آشنایی با تراشه ۷۴۱۸۱
۳	۱.۲	بررسی و انتخاب روش طراحی
۴	۲.۲	بلوک دیاگرام اولیه
۴	۳.۲	روند پیاده سازی
۵	۴.۲	شماتیک نهایی مدار
۶	۵.۲	بررسی Test Case ها
۹	۳	ساخت مدار داخلی ALU
۹	۱.۳	بررسی و انتخاب روش طراحی
۱۱	۲.۳	بلوک دیاگرام اولیه
۱۱	۳.۳	روند پیاده سازی
۱۳	۴.۳	شماتیک نهایی مدار
۱۴	۵.۳	بررسی Test Case ها
۱۷	۴	نتیجه گیری

۱ مقدمه و تعریف مسئله

واحد محاسبات و منطق (ALU) یکی از اساسی‌ترین بخش‌های هر پردازنده است که وظیفه انجام عملیات حسابی (مانند جمع و تفریق) و منطقی (مانند AND، OR، NOT و...) را بر روی داده‌های ورودی بر عهده دارد. در این آزمایش، هدف اصلی آشنایی با عملکرد تراشه‌ی 74181 به‌عنوان یک ALU چهاربیتی و نیز نحوه‌ی کنترل عملیات آن از طریق خطوط دستور (M_0 تا M_2) می‌باشد. همچنین با استفاده از قطعات جانبی نظیر ثبات‌های 74175، مالتی‌پلکسرها (MUX) و گیت‌های پایه، مداری طراحی می‌شود که قادر به ذخیره‌سازی داده‌ها و اجرای فرمان‌های مختلف بر روی دو ورودی A و B باشد.

بر اساس جدول عملیات ارائه‌شده، با اعمال کدهای متفاوت به خطوط M_0 تا M_2 ، عملیات‌هایی نظیر بارگذاری مقدار ورودی در ثبات A یا B، انتقال محتوا، پاک‌سازی، NOT، AND و جمع انجام می‌گیرد. برای شبیه‌سازی و تست مدار از نرم‌افزار Proteus استفاده می‌شود. ورودی‌های مدار شامل خطوط داده D_0 تا D_3 ، خطوط فرمان M_0 تا M_2 ، یک کلید push-button برای ورودی ساعت (clock) و یک کلید دیگر برای بازنشانی (Reset) هستند. خروجی‌های مدار نیز محتوای ثبات‌های A و B و همچنین خروجی نهایی ALU را نشان می‌دهند که صحت عملکرد مدار را تأیید می‌کنند.

در بخش دیگری از آزمایش، یک ALU چهاربیتی به‌صورت داخلی با استفاده از تراشه‌های 74181 (به‌عنوان هسته‌ی محاسباتی)، ثبات‌ها، مالتی‌پلکسرها و گیت‌های پایه ساخته می‌شود تا مفاهیم طراحی مدارهای ترکیبی و ترتیبی در کنار یکدیگر مرور گردد. این پیاده‌سازی علاوه بر آشنایی با قطعات، درک بهتری از نحوه‌ی عملکرد داخلی واحدهای محاسباتی در ریزپردازنده‌ها فراهم می‌آورد.

در ادامه، ابتدا با تراشه‌ی 74181 و پایه‌های آن آشنا شده، سپس مدار تست و در نهایت ساختار داخلی ALU چهاربیتی مورد بررسی قرار می‌گیرد.

۲ آشنایی با تراشه ۷۴۱۸۱

۱.۲ بررسی و انتخاب روش طراحی

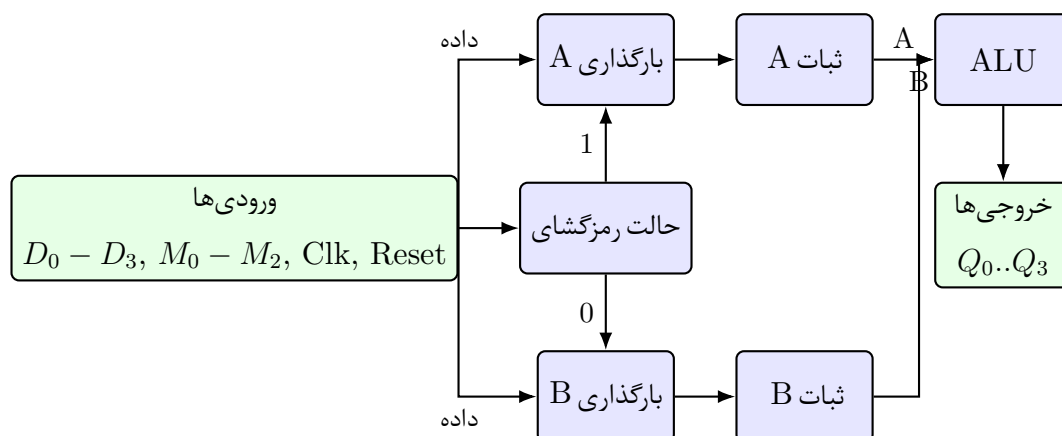
مداری که قصد طراحی آنرا داریم، مطابق جدول ۱ کار میکند:

جدول ۱: عملیات صورت گرفته در مدار بر حسب ورودی‌های $M_2 - M_0$

عملیات	M_0	M_1	M_2
$A \leftarrow D_3 D_2 D_1 D_0$	۰	۰	۰
$B \leftarrow D_3 D_2 D_1 D_0$	۱	۰	۰
$A \leftarrow A$	۰	۱	۰
$A \leftarrow B$	۱	۱	۰
$clear(A)$	۰	۰	۱
$A \leftarrow not(A)$	۱	۰	۱
$A \leftarrow and(A, B)$	۰	۱	۱
$A \leftarrow add(A, B)$	۱	۱	۱

برای پیاده‌سازی مدار موردنظر مطابق جدول عملیات، نیاز به ترکیبی از مدارهای ترتیبی و ترکیبی داریم. با توجه به اینکه عملیات‌های مختلف نیازمند ذخیره‌سازی موقت داده‌ها و انجام محاسبات بر روی آن‌ها هستند، استفاده از ثبات‌های 74175 (که دارای پایه‌های مجزای بارگذاری و پاک‌سازی هستند) به‌عنوان حافظه‌های موقت A و B انتخاب شده‌اند. این ثبات‌ها داده‌های ورودی را در لبه‌ی بالارونده‌ی کلاک ذخیره کرده و تا دریافت فرمان بعدی، آن‌را نگه‌داری می‌کنند. هسته‌ی اصلی محاسبات، تراشه‌ی 74181 است که قابلیت انجام عملیات جمع، AND و NOT را به‌صورت سخت‌افزاری فراهم می‌کند؛ همچنین با تنظیم مناسب پایه‌های انتخاب (S0 تا S3 و M)، می‌توان داده‌های ورودی را بدون تغییر از آن عبور داد که برای اجرای دستورات انتقال $A \leftarrow A$ و $A \leftarrow B$ مفید است.

۲.۲ بلوک‌دیاگرام اولیه



شکل ۱.۲.۲: بلوک‌دیاگرام واحد محاسبات و منطق (ALU) با ثبات‌های A و B و رمزگشای حالت

۳.۲ روند پیاده‌سازی

با توجه به اینکه دستورات توسط سه خط M_0 تا M_2 مشخص می‌شوند، در حالی که تراشه‌ی 74181 دارای چهار خط انتخاب ($S_0 - S_3$) و یک خط حالت (M) است، نمی‌توان مستقیماً خطوط دستور را به پایه‌های ALU متصل کرد. بنابراین، یک بلوک رمزگشای حالت (Mode Decoder) متشکل از مالتی‌پلکسرهای (MUX) و گیت‌های پایه در نظر گرفته می‌شود تا کدهای سه‌بیتی را به سیگنال‌های کنترلی مناسب تبدیل کند. این رمزگشا وظیفه دارد:

۱. سیگنال بارگذاری (Load) را برای ثبات A یا B به‌ترتیب برای کدهای 000 و 001 فعال کند،
۲. در صورت نیاز، داده‌های ورودی D یا خروجی ALU را برای بارگذاری در ثبات A انتخاب کند،
۳. کد عملیاتی متناظر با جمع، AND و NOT را به پایه‌های انتخاب تراشه‌ی 74181 اعمال نماید.

برای دستور پاک‌سازی (100)، به‌جای اعمال عملیات به ALU، مقدار صفر به‌صورت مستقیم به ورودی ثبات A اعمال می‌شود (یا با استفاده از گیت‌های منطقی، خروجی ALU صفر می‌شود). همچنین دو کلید فشاری (Push-Button) برای تأمین سیگنال‌های کلاک و Reset در نظر گرفته شده‌اند تا کاربر بتواند به‌صورت دستی، گام‌به‌گام عملیات را پیش برده و در صورت نیاز مدار را به حالت اولیه بازگرداند. خروجی‌های

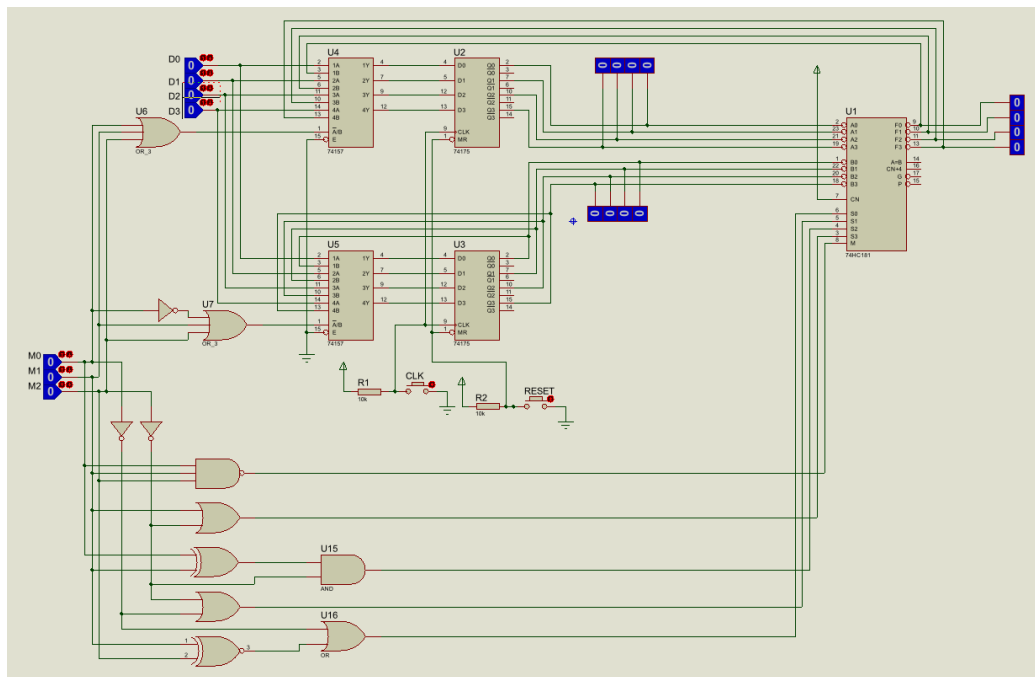
مدار شامل محتوای ثبات‌های A و B و همچنین خروجی نهایی ALU هستند. این روش طراحی، ضمن سادگی، امکان تست تمامی هشت دستور خواسته‌شده را با کمترین پیچیدگی فراهم می‌کند.

Table 2: All 32 functions of the 74181 ALU

Selects ($S_3S_2S_1S_0$)	Logic ($M = 1$)	Arithmetic ($M = 0, C_n = 0$)
0000	$\overline{A}$	A
0001	$\overline{A + B}$	$A + B$
0010	$\overline{A} \cdot B$	$A + \overline{B}$
0011	0	-1 (all ones)
0100	$\overline{A \cdot B}$	$A + A\overline{B}$
0101	$\overline{B}$	$(A + B) + A\overline{B}$
0110	$A \oplus B$	$A - B - 1$
0111	$A \cdot \overline{B}$	$A\overline{B} - 1$
1000	$\overline{A} + B$	$A + AB$
1001	$\overline{A \oplus B}$	$A + B$
1010	B	$(A + \overline{B}) + AB$
1011	$A \cdot B$	$AB - 1$
1100	1	$A + A$
1101	$A + \overline{B}$	$(A + B) + A$
1110	$A + B$	$(A + \overline{B}) + A$
1111	A	$A - 1$

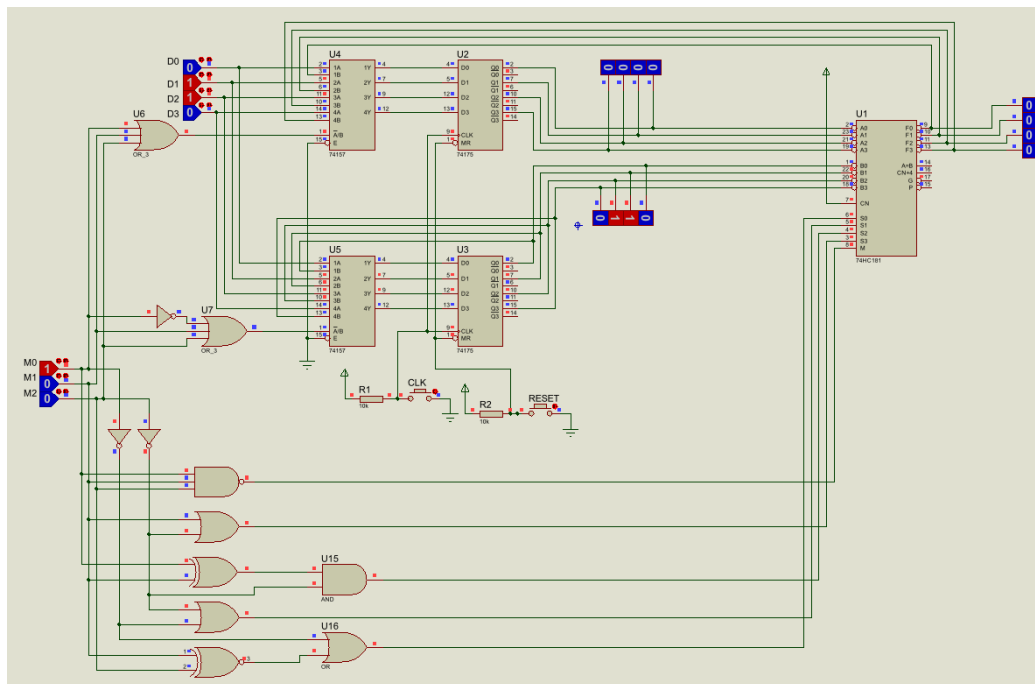
در جدول ۲ تمام قابلیت‌های تراشه ۷۴۱۸۱ به ازای مقادیر مختلف Select و M را می‌بینیم. با استفاده از حالت‌های مورد نیاز خود از این جدول، توابعی را که ورودی پایه‌های M و S_0 تا S_3 را می‌سازند، بدست می‌آوریم. در این مرحله با عملیات جبری ساده‌ای مواجه هستیم که بررسی جزئی آن ارزش زیادی ندارد.

۴.۲ شماتیک نهایی مدار

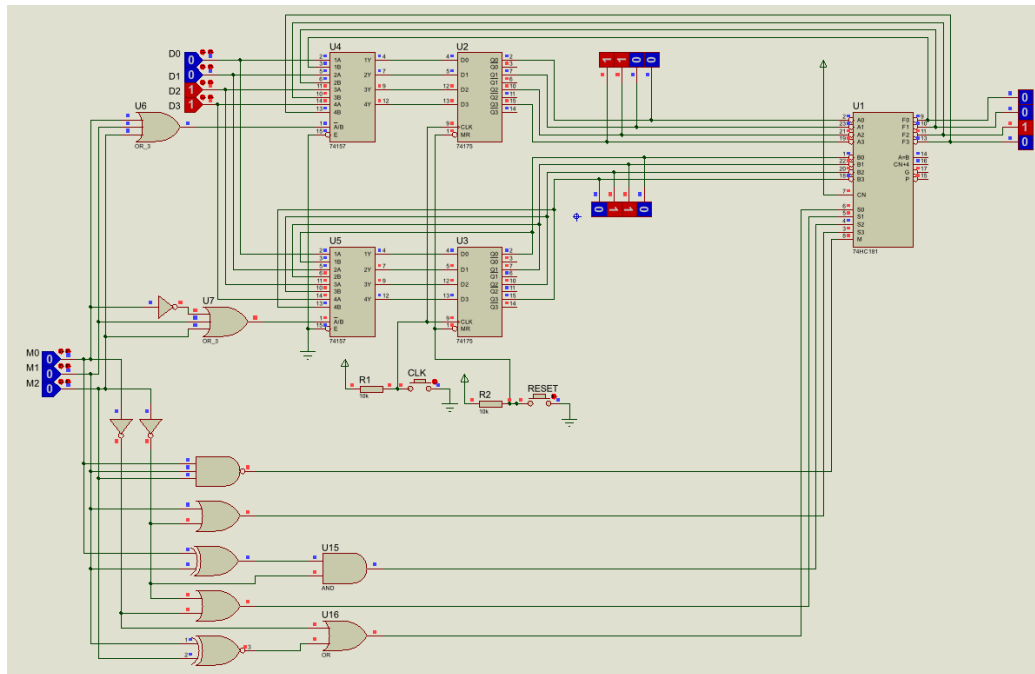


شكل ١.٤.٢

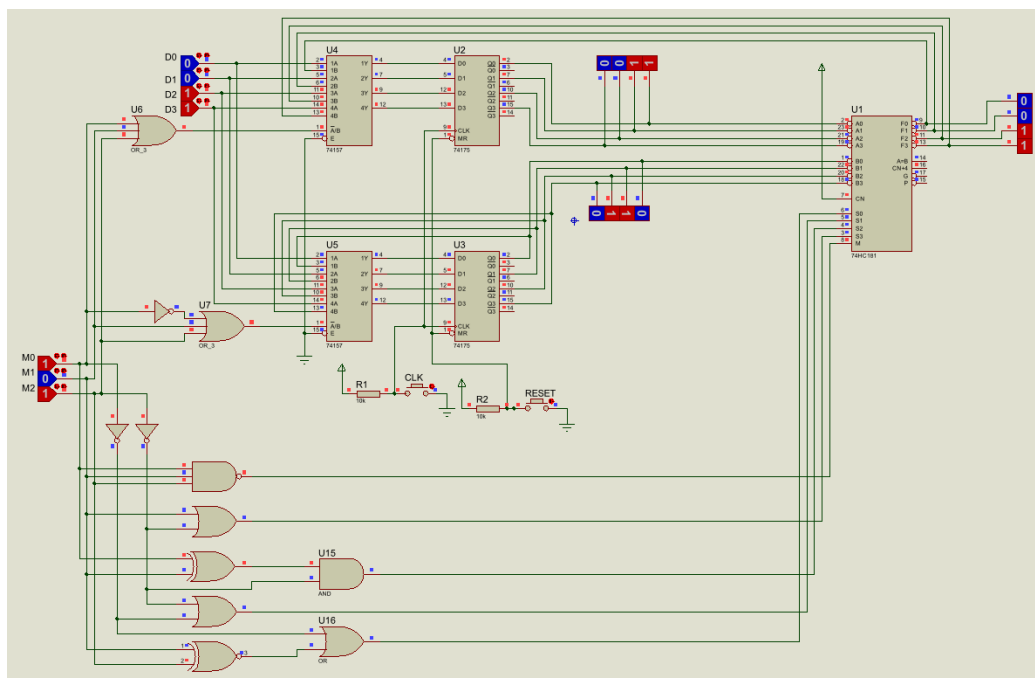
۵.۲ بررسی Test Case ها



شکل ۱.۵.۲: بارگذاری B



شکل ۲.۵.۲: بارگذاری A



شکل ۳.۵.۲: NOT کردن B

۳ ساخت مدار داخلی ALU

۱.۳ بررسی و انتخاب روش طراحی

طبق دستور کار، ALU ما مطابق جدول ۳ عمل می کند:

جدول ۳: جدول عملکرد ALU چهاربیتی (مبنای طراحی)

توضیح	عملیات	C_{in}	S_0	S_1	S_2	S_3
انتقال A	$F = A$	۰	۰	۰	۰	۰
افزایش (اینکریمنت) A	$F = A + 1$	۱	۰	۰	۰	۰
جمع	$F = A + B$	۰	۱	۰	۰	۰
جمع با کری	$F = A + B + 1$	۱	۱	۰	۰	۰
تفریق با قرض	$F = A + \overline{B}$	۰	۰	۱	۰	۰
تفریق	$F = A + \overline{B} + 1$	۱	۰	۱	۰	۰
کاهش (دیکرمنت) A	$F = A - 1$	۰	۱	۱	۰	۰
انتقال A	$F = A$	۱	۱	۱	۰	۰
AND	$F = A \wedge B$	×	۰	۰	۱	۰
OR	$F = A \vee B$	×	۱	۰	۱	۰
XOR	$F = A \oplus B$	×	۰	۱	۱	۰
متمم A	$F = \overline{A}$	×	۱	۱	۱	۰
شیفت به راست A در F	$F = shr A$	×	×	×	۰	۱
شیفت به چپ A در F	$F = shl A$	×	×	×	۱	۱

برای تحقق یک ALU چهاربیتی، رویکرد این آزمایش بر پایه‌ی بهره‌گیری از تراشه‌های آماده به‌جای طراحی تماماً از صفر با گیت‌های مجزا استوار شده است. این انتخاب، افزون بر سادگی پیاده‌سازی، با امکانات کتابخانه‌ی شبیه‌ساز Proteus و تجهیزات در دسترس آزمایشگاه هماهنگی کامل دارد. اجزاء اصلی که زیربنای مدار را تشکیل می‌دهند، عبارتند از:

- جمع‌کننده‌ی چهاربیتی 74283 که نقش هسته‌ی محاسباتی عملیات جمع و تفریق را ایفا می‌کند؛
- مالتی‌پلکسرهای 74153 (شامل دو مالتی‌پلکسر چهاربه‌یک با پایه‌های Select مشترک) که کار انتخاب عملوندها و هدایت نتایج بخش‌های گوناگون را بر عهده دارند؛

- گیت‌های مجزای AND، OR، XOR و NOT برای تولید خروجی‌های منطقی و سیگنال‌های کمکی مانند متمم‌های A و B.

مبنای این انتخاب، قابلیت‌های ذاتی تراشه‌ی 74283 در انجام جمع چهاربیتی با در نظر گرفتن کری ورودی و خروجی است. با تغییر مقدار ورودی دوم این تراشه (به کمک مالتی‌پلکسرها)، می‌توان مجموعه‌ی کامل عملیات حسابی مورد نظر در جدول عملکرد را - بدون نیاز به مدارهای مجزای جمع و تفریق - پیاده‌سازی کرد. در این چیدمان، هر زیرمجموعه به‌طور مستقل طراحی و سپس خروجی‌های آن‌ها از طریق یک لایه‌ی مالتی‌پلکسر نهایی - بر اساس خطوط انتخاب S_2 و S_3 - به‌عنوان خروجی نهایی F برگزیده می‌شوند.

به این ترتیب، کل مدار به سه بخش مستقل تفکیک می‌گردد:

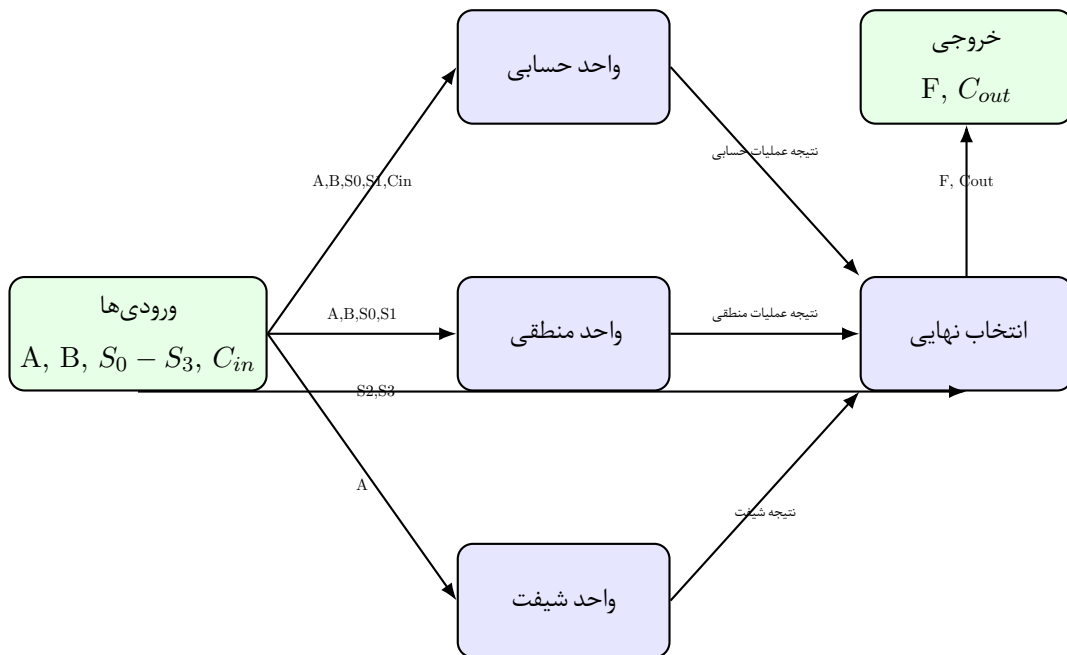
۱. واحد محاسباتی (شامل جمع، تفریق، افزایش و کاهش)؛

۲. واحد منطقی (شامل AND، OR، XOR و NOT)؛

۳. واحد شیفت (شامل شیفت منطقی به راست و چپ).

خروجی هر یک از این سه واحد، به‌همراه خطوط انتخاب S_2 ، S_3 ، به مالتی‌پلکسرهای نهایی داده می‌شود تا در هر لحظه، تنها یکی از آن‌ها به‌عنوان خروجی F ظاهر گردد. شرح دقیق نحوه‌ی تحقق هر یک از این زیرمجموعه‌ها در بخش «رند پیاده‌سازی» ارائه شده است. جدول ۳ نیز مرجع کاملی از تمام عملیات قابل اجرا توسط این مدار به‌دست می‌دهد و مشخص می‌کند که چگونه ترکیب خطوط انتخاب، نوع عملیات (حسابی، منطقی یا شیفت) و زیرحالت آن را تعیین می‌کند.

۲.۳ بلوک‌دیاگرام اولیه



شکل ۱.۲.۳: بلوک‌دیاگرام ساده‌شده‌ی مدار داخلی ALU چهاربیتی

۳.۳ روند پیاده‌سازی

پس از تثبیت ساختار کلی، نوبت به ترجمه‌ی بلوک‌های مفهومی به مداری قابل اجرا در Proteus رسید. پیاده‌سازی به صورت گام‌به‌گام و با تفکیک کامل سه زیرمجموعه انجام شد تا هر بخش به طور مستقل تست و تأیید گردد؛ سپس در مرحله‌ی نهایی، خروجی‌ها به یکدیگر متصل شده و لایه‌ی انتخاب نهایی، پاسخ نهایی مدار را رقم زد.

زیرمجموعه‌ی حسابی

مهم‌ترین بخش از نظر پیچیدگی، واحد حسابی بود. ایده‌ی اصلی این بود که به جای طراحی مجزای جمع‌کننده و تفریق‌کننده، از یک جمع‌کننده‌ی چهاربیتی (تراشه‌ی 74283) استفاده شود و با تغییر یکی از عملوندهای آن، تمام هشت حالت حسابی موردنظر پوشش داده شود. به عبارت دیگر، به جای آنکه مستقیماً B را به پایه‌های دوم جمع‌کننده متصل کنیم، تصمیم گرفته شد که مقدار ورودی دوم (X) توسط یک جفت مالتی‌پلکسر 74153 از میان چهار گزینه‌ی $0, B, \bar{B}$ و 1 انتخاب گردد. این انتخاب بر اساس خطوط S_0 و

S_1 انجام می‌شود؛ به‌طوری که برای مثال، وقتی $S_1 S_0 = 00$ باشد، $X=0$ انتخاب شده و جمع‌کننده صرفاً A را به‌همراه C_{in} عبور می‌دهد (که معادل عملیات انتقال یا افزایش A است). برای $S_1 S_0 = 01$ ، $X=B$ و عملیات جمع معمولی حاصل می‌شود؛ با $S_1 S_0 = 10$ ، $X=\bar{B}$ و تفریق با قرض پدید می‌آید؛ و نهایتاً با $S_1 S_0 = 11$ ، مقدار $X=1$ باعث می‌شود که عملیات کاهش A (یا در حالت خاص، انتقال مجدد) محقق گردد.

چهار مالتی‌پلکسر مجزا (که در دو تراشه‌ی 74153 جای گرفته‌اند) هر یک مسئول انتخاب یک بیت از X هستند. پایه‌های انتخاب این مالتی‌پلکسرها به‌صورت مشترک به S_0 و S_1 متصل شدند تا رفتار هماهنگی داشته باشند. در خروجی جمع‌کننده، بیت‌های حاصل از جمع (F_0 تا F_3) به‌همراه کری خروجی (C_{out}) ظاهر می‌شوند؛ اما این خروجی‌ها هنوز خروجی نهایی نیستند، زیرا ممکن است مدار در حالت منطقی یا شیفت باشد و نباید نتیجه‌ی حسابی نمایش داده شود. به‌همین دلیل، خروجی جمع‌کننده به یکی از ورودی‌های مالتی‌پلکسر نهایی هدایت می‌شود تا در صورت لزوم، توسط خطوط S_2 و S_3 انتخاب گردد. زیرمجموعه‌ی منطقی

بخش منطقی از چهار نوع گیت پایه تشکیل شد: AND، OR، XOR و NOT. برای هر چهار بیت، به‌طور موازی عملیات بیتی اعمال گردید؛ برای مثال، چهار گیت AND جداگانه خروجی $A \wedge B$ را بیت‌به‌بیت تولید کردند. سپس یک جفت مالتی‌پلکسر دیگر (از همان نوع 74153) وظیفه‌ی انتخاب یکی از این چهار نتیجه‌ی منطقی را بر عهده گرفت. خطوط انتخاب S_0 و S_1 دوباره در اینجا نقش کلیدی ایفا کردند؛ به‌طوری که با کد $S_1 S_0 = 00$ ، خروجی AND؛ با 01 ، خروجی OR؛ با 10 ، خروجی XOR و با 11 ، خروجی NOT (متمم A) به‌عنوان نتیجه‌ی منطقی برگزیده می‌شوند. این نتیجه نیز به‌مانند بخش حسابی، به ورودی دیگری از مالتی‌پلکسر نهایی فرستاده شد تا در انتخاب نهایی سهیم باشد.

توجه به این نکته ضروری است که در حالت منطقی، مقدار C_{in} نقشی ندارد و پایه‌ی مربوطه بی‌اثر است. همچنین در شبیه‌سازی، صحت عملکرد گیت‌ها با چندین مجموعه داده‌ی تصادفی آزموده شد تا از عدم تداخل یا اشتباه در سیم‌کشی اطمینان حاصل گردد. زیرمجموعه‌ی شیفت

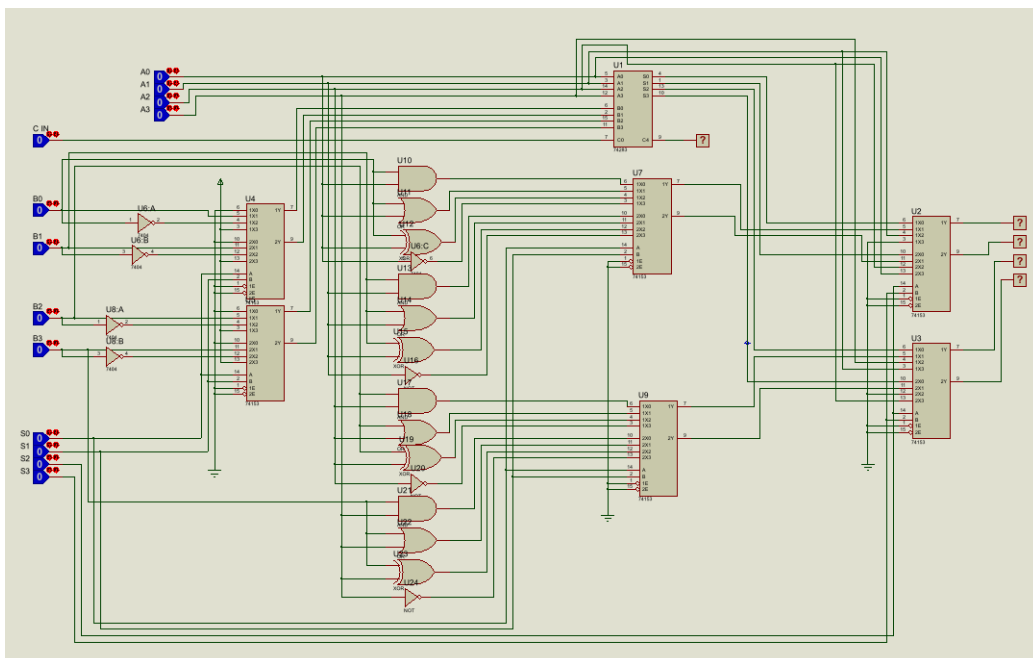
برخلاف دو بخش قبلی، واحد شیفت نیازی به تراشه‌ی خاصی نداشت و صرفاً با سیم‌کشی مستقیم پیاده‌سازی شد. برای شیفت به راست، خروجی F به‌گونه‌ای سیم‌کشی شد که بیت A_1 به F_0 ، A_2 به F_1 ، A_3 به F_2 و بیت 0 (زمین) به F_3 متصل گردد. به عبارت دیگر، یک جابه‌جایی یک‌بیتی به سمت راست با ورود صفر از سمت چپ انجام شد. برای شیفت به چپ نیز برعکس این ترتیب به کار رفت: A_0 به F_1 ، A_1 به F_2 ، A_2 به F_3 و زمین به

F_0 وصل شد. بدیهی است که در هر دو حالت، بیت‌های خارج شده از محدوده‌ی چهاربیتی به کلی حذف شدند (چون ALU ظرفیت نگهداری بیت‌های خارج شده را ندارد). این دو نتیجه‌ی شیفت، به ترتیب به دو ورودی باقی‌مانده‌ی مالتی‌پلکسرهای نهایی متصل شدند تا در انتخاب نهایی، زمانی که $S_3S_2 = 10$ یا 11 است، یکی از آن‌ها به عنوان خروجی برگزیده شود.

لایه‌ی انتخاب نهایی

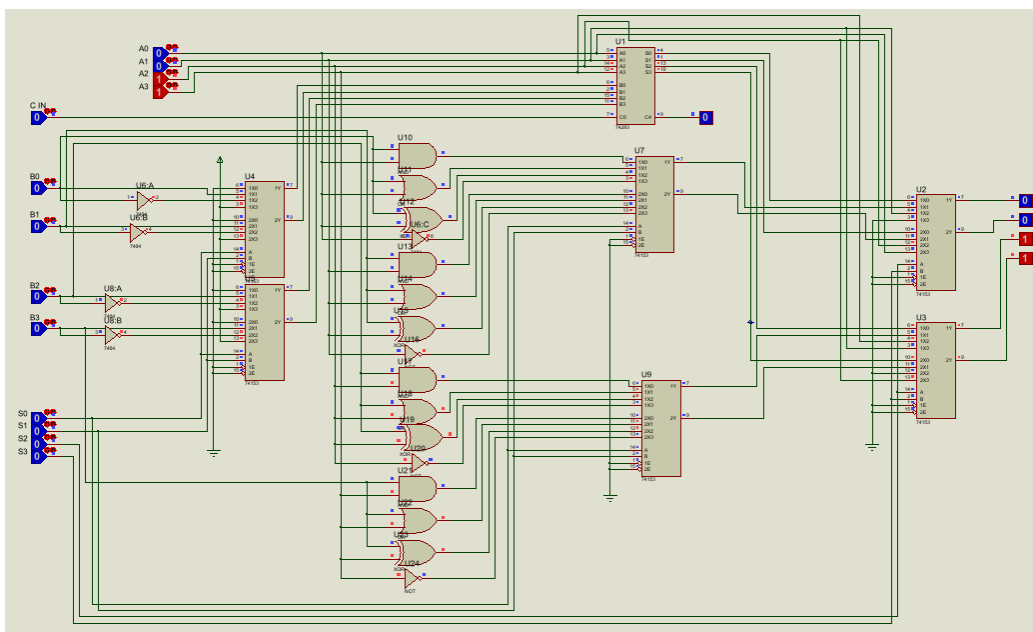
دو مالتی‌پلکسر 74153 پایانی، نقش کلیدی در تعیین خروجی نهایی داشتند. ورودی‌های اول این مالتی‌پلکسرها به نتیجه‌ی واحد حسابی، ورودی‌های دوم به نتیجه‌ی واحد منطقی، و ورودی‌های سوم به نتیجه‌ی شیفت راست و ورودی‌های چهارم به نتیجه‌ی شیفت چپ متصل شدند. پایه‌های انتخاب این مالتی‌پلکسرها به خطوط S_2 و S_3 وصل گردیدند تا بر اساس کد این دو خط، یکی از چهار مسیر برگزیده شود؛ به طوری که با $S_3S_2 = 00$ ، خروجی حسابی؛ با 01 ، خروجی منطقی؛ با 10 ، شیفت راست و با 11 ، شیفت چپ به عنوان خروجی نهایی F ظاهر می‌شود. کری خروجی (C_{out}) نیز مستقیماً از خروجی جمع‌کننده گرفته شد و در حالت‌های غیر حسابی، این سیگنال بی‌معنا تلقی گردید.

۴.۳ شماتیک نهایی مدار

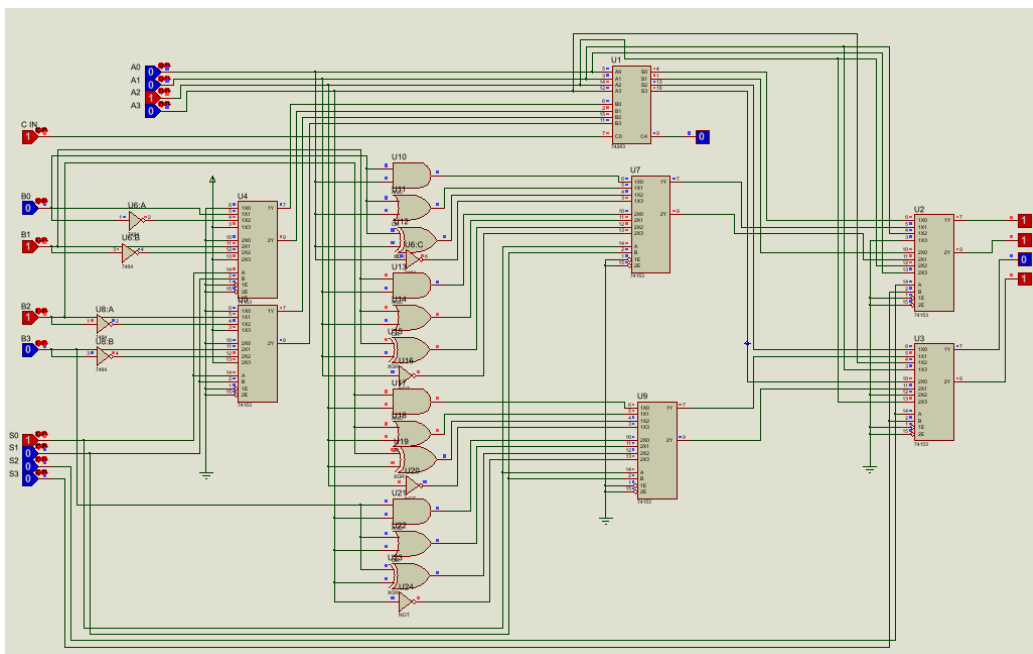


شکل ۱.۴.۳

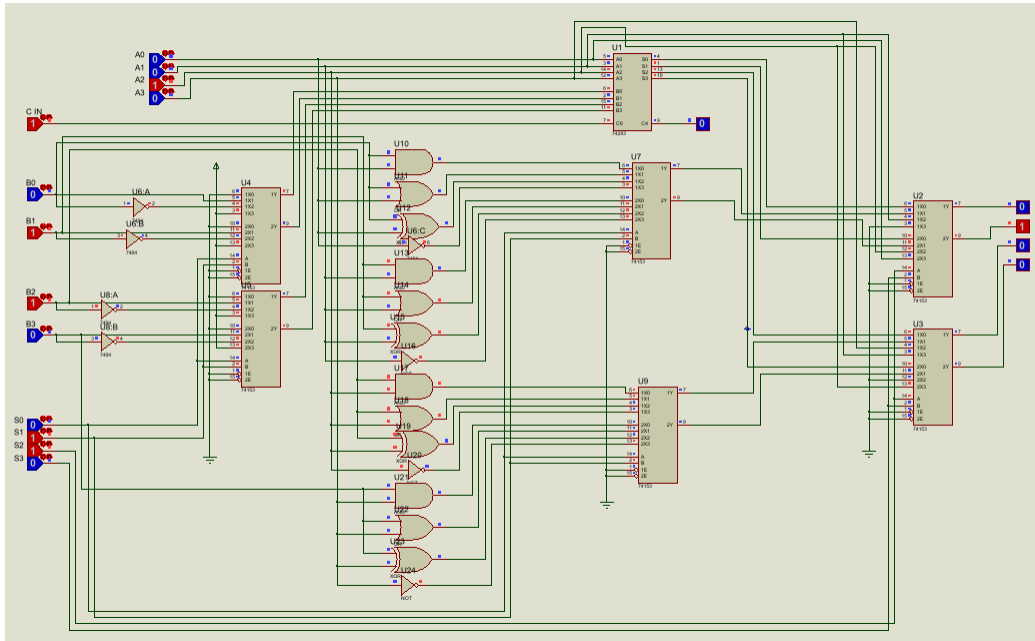
۵.۳ بررسی Test Case ها



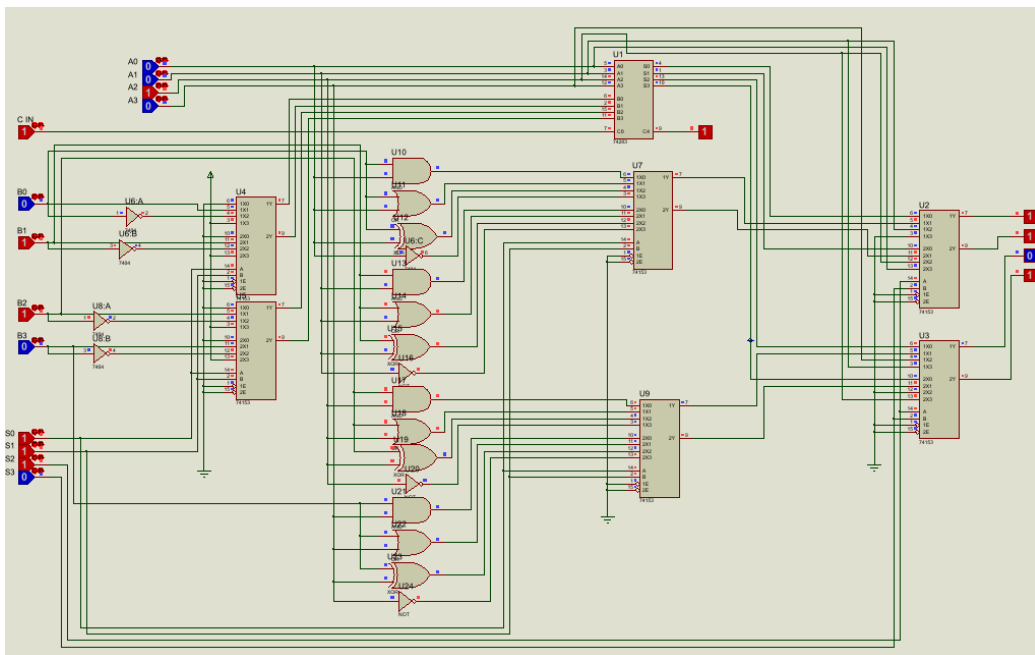
شکل ۱.۵.۳: انتقال A



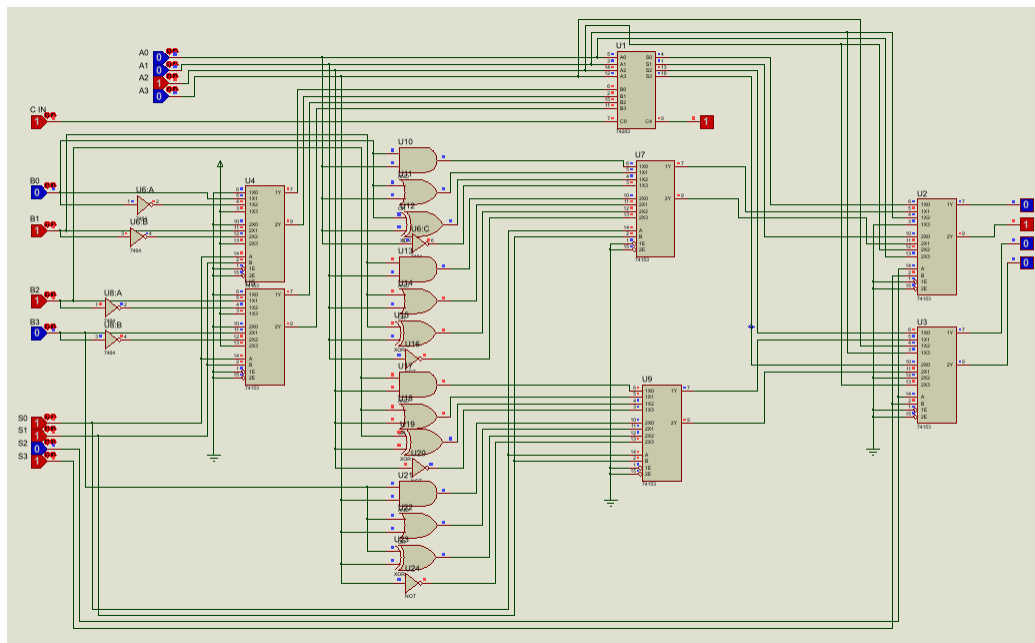
شکل ۲.۵.۳: $A + B$



شکل ۳.۵.۳: $A \oplus B$



شکل ۴.۵.۳: $\bar{A}$



شکل ۵.۵.۳: شیفت به راست A

۴ نتیجه‌گیری

در این آزمایش، با دو رویکرد متفاوت برای پیاده‌سازی یک واحد محاسبات و منطق چهاربیتی آشنا شدیم. در بخش نخست، با تراشه‌ی تخصصی 74181 کار کردیم که تمام عملیات حسابی و منطقی را درون یک بسته‌ی واحد ارائه می‌دهد و نشان داد که چگونه با اعمال کدهای مناسب به پایه‌های انتخاب، می‌توان به سرعت و بدون نیاز به سیم‌کشی گسترده، به خروجی مطلوب دست یافت. در بخش دوم، با کنار گذاشتن این تراشه‌ی همه‌کاره، گام به گام یک ALU را از قطعات ساده‌تر (جمع‌کننده، مالتی‌پلکسر و گیت‌های پایه) طراحی و پیاده‌سازی کردیم. این تجربه، درک عمیق‌تری از نحوه‌ی عملکرد داخلی این واحدهای پرکاربرد در پردازنده‌ها فراهم آورد و نشان داد که حتی پیچیده‌ترین عملیات‌ها نیز در نهایت به ترکیبی از عملیات‌های ساده‌تر قابل‌بازایی هستند. از مهم‌ترین دستاوردهای این آزمایش، آشنایی عملی با مفاهیمی چون انتخاب عملوند، مدیریت کری ورودی/خروجی، تفکیک وظایف میان زیرمجموعه‌های حسابی و منطقی، و نقش مالتی‌پلکسرها در هدایت داده‌ها بود. همچنین، تجربه‌ی کار با تراشه‌های مختلف در محیط شبیه‌ساز Proteus و مقایسه‌ی نتایج شبیه‌سازی با محاسبات دستی، مهارت در عیب‌یابی و راستی‌آزمایی مدارهای دیجیتال را به‌طور محسوسی افزایش داد. در طول پیاده‌سازی، با چالش‌هایی نظیر هماهنگ‌سازی پایه‌های انتخاب، مدیریت هم‌زمان چندین مالتی‌پلکسر و اطمینان از اعمال صحیح کری ورودی مواجه شدیم که همگی با بررسی مجدد جدول عملکرد و تطبیق دقیق سیم‌کشی با نقشه‌ی طراحی، مرتفع گردیدند. خوشبختانه، با رعایت اصول اولیه، مدار در نخستین تلاش به‌درستی کار کرد و تمام آزمون‌ها را با موفقیت پشت سر گذاشت.